ISE-A4

End-to-End Vertical Slice and Automated Verification

Two-Stage Clinic Appointment and Arrival-Confirmation System

Field	Details
Course	Intelligent Software Engineering
Assessment	ISE-A4
Maximum marks	5
Project continuity	ISE-A1 opportunity → ISE-A2 domain model → ISE-A3 repository investigation → ISE-A4 vertical slice
Report date	10 November 2026
Implementation version	ise-a4-slice-v1.0
Evidence snapshot	ise-a4-v1.0

Selected vertical slice

A patient signs in, creates a future appointment window, checks in during that window, receives one queue ticket, and an authorised receptionist views and calls the ticket.

System boundary

This is a coursework prototype for synthetic data. It is not a production clinic system and does not provide diagnosis, triage, medical-record functions or emergency-care routing.

Prepared as the A4 implementation and verification record for the same clinic appointment project developed across the earlier ISE assessments.

Contents

- 1. Executive summary
- 2. Project continuity and selected slice
- 3. Requirement traceability and task contract
- 4. Interface and API design
- 5. Architecture and end-to-end call path
- 6. Authorization, state change and recovery controls
- 7. Automated verification and execution evidence
- 8. Usability and error-behaviour evidence
- 9. Review record and limitations
- 10. Rollback and recovery guide
- 11. AI-use disclosure
- 12. Platform result summary
- Appendix A. Changed files and version identity
- Appendix B. Reproduction commands

Reading note

The report distinguishes automated checks that were executed from desk-based task walkthroughs. The test results are reproducible synthetic examples; they are not evidence of measured outcomes in a real clinic.

1. Executive summary

This A4 work implements a bounded end-to-end slice of the clinic appointment project: a patient authenticates, creates an appointment with a flexible arrival window, checks in at the clinic, receives one queue ticket, and an authenticated receptionist views and calls that ticket. The browser interface is served by FastAPI; SQLite stores users, sessions, appointments, tickets and idempotency responses. Passwords are stored as PBKDF2 hashes, bearer sessions are stored as token hashes, patient ownership and receptionist roles are checked on the server, and ticket calls use an If-Match version to reject stale updates. Check-in is transactional and uses an idempotency key so an uncertain retry returns the original ticket rather than creating a duplicate. The time interval is half-open: the start is accepted and the end is rejected.

The combined A2 and A4 automated run completed with 60 tests passing: 36 domain-model tests and 24 API integration/flow tests. Coverage includes authorization, ownership, invalid time windows, early and end-boundary check-in, duplicate requests, persistence, missing version headers and stale ticket versions. Synthetic data and a fixed clock make these tests deterministic. The interface route was smoke-tested, and scripted patient/receptionist task walkthroughs were documented. These walkthroughs are not a substitute for moderated testing with real representative users, and no independent human peer review or clinic deployment validation is claimed. The slice excludes clinical decisions, no-show automation, external messaging and medical-record integration.

Summary length

229 words. The platform requirement is 150–300 words.

2. Project continuity and selected slice

The project is carried forward from A1 and A2 rather than introducing a new problem. A1 framed the opportunity: appointment booking alone does not confirm that a patient has physically arrived, while manual queue coordination can create extra work. A2 defined the domain vocabulary and state rules, including an appointment moving from CONFIRMED to CHECKED_IN and a queue ticket beginning in WAITING. A4 implements one complete, bounded path through that model.

2.1 Why this slice was selected

The check-in-to-queue handoff is a valuable slice because it connects a patient-facing action to a clinic-facing result. A patient gets immediate confirmation that arrival has been registered; reception staff see an explicit waiting ticket and can call it through an authorised screen. The slice is small enough to review while still crossing the browser, API, authorization logic, domain state and persistence layers.

2.2 Included scope

- Patient sign-in using a synthetic demonstration account.
- Creation of a non-emergency appointment with a future arrival window and clinic code.
- Patient check-in only within the configured window using a half-open interval [start, end).
- Atomic update of appointment state and creation of one queue ticket.
- Receptionist sign-in, queue viewing and version-checked ticket calling.
- Visible success and failure feedback with safe retry for an uncertain check-in request.

2.3 Explicit non-goals

- No diagnosis, prescription, triage, emergency handling or medical-record storage.
- No automatic no-show marking, waitlist allocation, queue prioritisation or doctor scheduling optimisation.
- No integration with hospital HIS/EMR, payment, SMS, WeChat, identity-provider or third-party kiosk hardware.
- No claim that the prototype reduces real waiting time, no-shows or clinic staffing cost.

3. Requirement traceability and task contract

The implementation is linked to the earlier project artefacts. The identifiers below are project trace identifiers used in this report; they do not replace the course task numbering.

Trace ID	Origin	A4 requirement	Acceptance evidence
A1-PROB-01	A1 problem/value brief	A patient must be able to confirm physical arrival, not merely hold an online booking.	Successful check-in changes the appointment to CHECKED_IN and returns a ticket.
A1-PROB-02	A1 workflow / stakeholder analysis	Reception needs a queue view of confirmed arrivals.	Receptionist-only queue endpoint lists WAITING and CALLED tickets.
A2-DOM-01	A2 domain glossary and state model	Appointment and ticket states use the A2 domain vocabulary.	Database/API responses use CONFIRMED, CHECKED_IN, WAITING and CALLED.
A2-INV-01	A2 time-window invariant	Arrival window is valid, timezone-aware and half-open.	Tests cover naive, reversed, too-early, start and end boundary cases.
A2-INV-02	A2 duplicate-ticket invariant	One appointment produces at most one ticket.	Unique appointment constraint and idempotent repeat tests.

Trace ID	Origin	A4 requirement	Acceptance evidence
A2-INV-03	A2 authorization boundary	Patients access only their own appointment; queue operations require staff role.	Cross-owner and role-based API tests return 403.
A2-STATE-01	A2 ticket state model	A WAITING ticket can be called only against the current version.	If-Match and stale-version tests return controlled errors.

3.1 Patient check-in contract

Contract item	Specification
Actor	Authenticated patient who owns the appointment.
Request	POST /api/appointments/{appointment_id}/check-in with Idempotency-Key header.
Preconditions	Appointment exists; user owns it; state is CONFIRMED; current server time is within [window_start, window_end); key is present.
Successful postconditions	One transaction sets CHECKED_IN and checked_in_at, inserts one WAITING ticket and stores the response for safe replay.
Failure behaviour	401 unauthenticated; 403 wrong owner/role; 404 unknown appointment; 409 invalid state/time boundary; 422 malformed or missing input.
Recovery	Retry the same logical request with the same Idempotency-Key. The stored ticket response is returned; no second ticket is issued.

3.2 Reception ticket-call contract

Contract item	Specification
Actor	Authenticated receptionist.
Request	POST /api/staff/tickets/{ticket_id}/call with If-Match: {version}.
Preconditions	Ticket exists; caller has receptionist role; supplied version equals stored version; ticket state is WAITING.
Postconditions	Ticket becomes CALLED, called_at is recorded and version increments by one.
Failure behaviour	401/403/404 for identity or lookup failures; 428 if version missing; 400 malformed version; 412 stale version; 409 invalid current state.
Recovery	Refresh the queue to obtain current state/version, then retry deliberately. No silent overwrite occurs.

4. Interface and API design

The interface is a single responsive HTML page served by FastAPI. It separates patient and reception tasks into visible sections, uses labelled inputs, provides status messages for success and failure, and shows a

queue number after check-in. The browser sends JSON to the API; it does not make authorization decisions itself.

Endpoint	Method	Authorized actor	Purpose / response
/	GET	Public	Serves the browser interface.
/health	GET	Public	Simple live-service health response.
/api/login	POST	Public credentials	Checks password and creates an expiring bearer session.
/api/appointments	POST	Patient	Creates a future appointment with clinic and arrival window.
/api/appointments/{id}	GET	Owner or receptionist	Returns appointment details subject to ownership/role.
/api/appointments/{id}/check-in	POST	Owning patient	Checks window, updates appointment and issues a queue ticket.
/api/staff/queue?clinic_code=...	GET	Receptionist	Lists active WAITING/CALLED tickets for a clinic.
/api/staff/tickets/{id}/call	POST	Receptionist	Calls a WAITING ticket using optimistic version check.

4.1 Interface feedback

- Success: appointment identifier, check-in confirmation, queue number and ticket state are displayed.
- Validation/error: the response message appears beside the relevant task; examples include “Check-in is not open yet” and “Arrival window has ended; ask reception for review”.
- Retry: a generated idempotency key is retained when the check-in request fails at the client/network boundary, so pressing retry does not create a new logical request.
- Accessibility basics: explicit labels are associated by proximity and input ids; dynamic status areas use aria-live="polite"; the page uses normal keyboard-focusable form controls and buttons.

4.2 API request examples

```
POST /api/appointments/{appointment_id}/check-in
Authorization: Bearer <session-token>
Idempotency-Key: 2d9b8...

POST /api/staff/tickets/{ticket_id}/call
Authorization: Bearer <reception-session-token>
If-Match: 1
```

5. Architecture and end-to-end call path

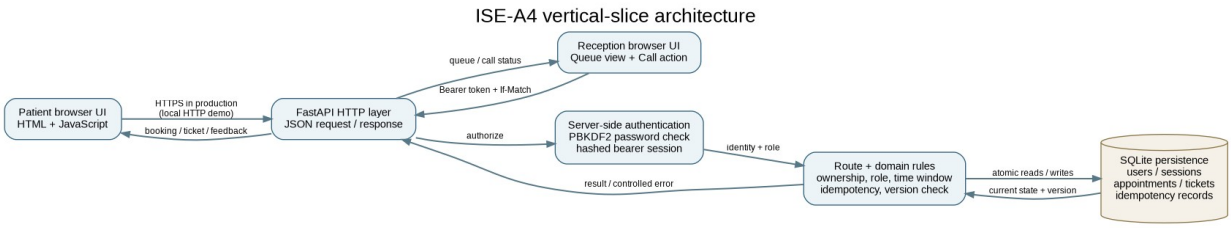


Figure 1. Browser, FastAPI, server-side authorization, domain checks and SQLite persistence.

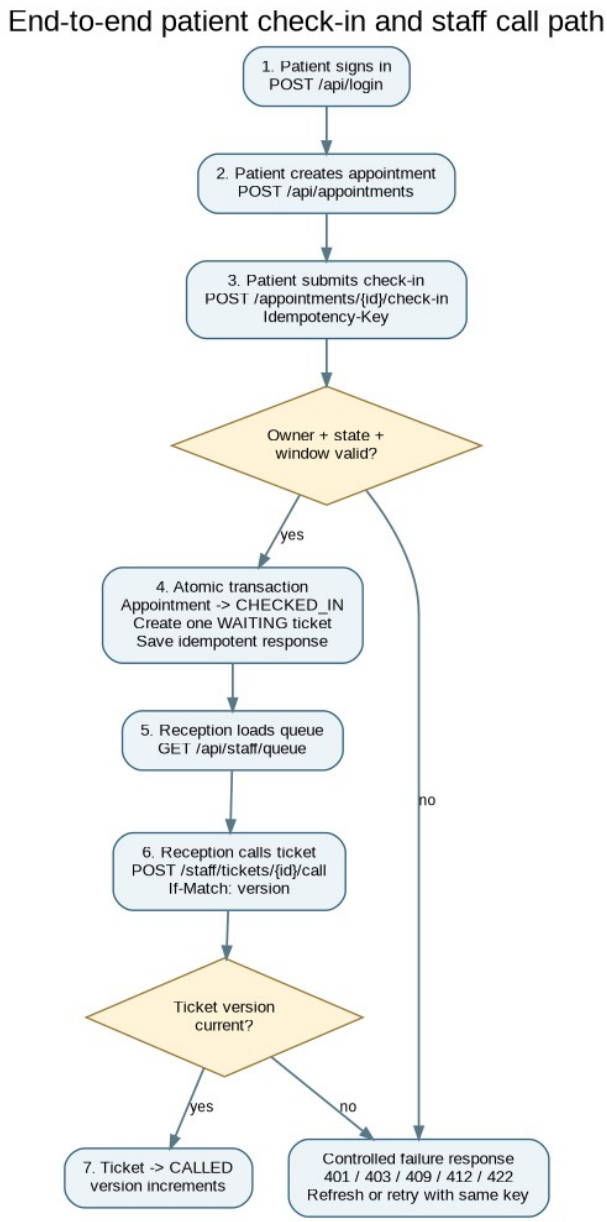


Figure 2. Patient check-in through receptionist ticket calling, including controlled failure paths.

The route handlers enforce role/ownership and state rules before writing data. SQLite constraints provide a second line of defence: appointment and ticket identifiers are primary keys, each appointment_id is unique in the tickets table, and each queue number is unique for a clinic/date. The check-in path uses an

explicit write transaction so the appointment state and ticket record are committed together or rolled back together.

6. Authorization, state change and recovery controls

6.1 Authentication and authorization

- Passwords are stored as PBKDF2-HMAC-SHA256 hashes with a random salt; plaintext passwords are not stored.
- Login issues a random bearer token. Only its SHA-256 hash is stored in the session table; the token expires after eight hours in this prototype.
- Every protected API route resolves the session on the server. Patient routes verify appointment ownership; staff queue and call routes require the receptionist role.
- The browser hides or displays sections for usability only. The server remains the enforcement boundary.

6.2 State and data change

Object	Before	Action	After	Consistency control
Appointment	CONFIRMED	Patient check-in in valid window	CHECKED_IN	Single transaction; checked_in_at recorded; version increments.
Queue ticket	Does not exist	Created during check-in	WAITING	Unique appointment_id; queue number unique per clinic/date.
Queue ticket	WAITING	Receptionist call with current version	CALLED	If-Match + conditional update; version increments.
Idempotency record	No key record	Successful check-in	Stored response	Unique (user_id, key); retry replays response.

6.3 Duplicate and stale request handling

A check-in request is protected by an Idempotency-Key. When the same patient repeats the same key, the API returns the already stored response. The database also makes appointment_id unique in the ticket table, preventing a second ticket even if a different key is supplied after check-in. Reception ticket calls use If-Match with a monotonically increasing version. A stale version receives HTTP 412 and must refresh before retrying.

6.4 Main failure and recovery matrix

Condition	API response	Expected state effect	Recovery / user feedback
No/expired session	401	No data mutation	Sign in again.
Wrong role or appointment owner	403	No data mutation	Use an authorized account; no sensitive record is disclosed.
Window not open / end reached	409	Appointment remains CONFIRMED; no ticket	Explain boundary and direct late arrivals to reception.
Invalid or missing fields	422	No data mutation	Correct the form input.
Duplicate/uncertain retry	200 stored response	Existing ticket retained	Reuse the same idempotency key.

Condition	API response	Expected state effect	Recovery / user feedback
Stale staff version	412	No stale overwrite	Refresh queue and use latest version.
Ticket already CALLED	409	No additional transition	Refresh and follow current ticket state.

7. Automated verification and execution evidence

Verification is layered. The 36 earlier A2 domain tests continue to check pure time-window, appointment-coherence, ticket uniqueness, no-show authorization and state-transition rules. The 24 A4 tests exercise the FastAPI routes and persistence using temporary SQLite databases, synthetic accounts and a fixed clock.

Layer	Evidence	Result
A2 domain/unit	36 tests from tests/test_clinic_model.py	36 passed
A4 API integration and flow	24 tests from tests/test_vertical_slice.py using FastAPI TestClient and temporary SQLite	24 passed
Combined regression	python -m unittest discover -s tests -v	60 passed; 0 failed
Runtime smoke	Uvicorn local server; GET /health and GET /	HTTP 200 for both; UI title found
Browser-level E2E	No automated real-browser journey recorded	Not claimed

7.1 Test coverage matrix

Risk / behaviour	Representative checks in the suite
Authentication	Invalid password; missing authentication.
Authorization / isolation	Patient cannot use staff queue/call; patient cannot access another patient's appointment.
Arrival boundaries	Too early; start boundary accepted; end boundary rejected; naive timestamps and reversed windows rejected.
Duplicate request	Same key replays same ticket; new key after check-in does not create another ticket.
State/data change	Check-in stores CHECKED_IN timestamp and one WAITING ticket; staff call changes ticket to CALLED.
Concurrency	Missing If-Match rejected; stale If-Match returns 412; repeated call rejected.
Persistence / failures	State persists after app instance recreation; unknown appointment returns 404; malformed requests return 422.

7.2 Raw output excerpt

```
Ran 60 tests in 3.852s
```

```
OK

Live server smoke:
GET /health  -> HTTP 200
GET /        -> HTTP 200
UI title/content check -> present
```

The complete verbose test output is retained in `results/a4/combined_test_run.txt`; environment details are in `results/a4/environment.txt`; summary counts are in `results/a4/test_summary.json`. The runtime used Python 3.13.5, FastAPI 0.128.2, Starlette 0.50.0, Pydantic 2.13.5, HTTPX 0.28.1 and Uvicorn 0.48.0. The lock file is `requirements-a4.lock`.

8. Usability and error-behaviour evidence

The patient and receptionist tasks were walked through using scripted representative personas. This is a desk-based scenario walkthrough, not moderated testing with actual patients or clinic employees.

Representative task	Expected behaviour	Recorded evidence
Patient books then checks in	Appointment appears as confirmed; check-in creates one waiting ticket and shows its queue number.	API create/check-in/state test passed; browser UI route returned the page.
Patient double-submits check-in	Same ticket is returned; no duplicate ticket.	Same-key replay and new-key replay API tests passed.
Patient arrives too early or at end boundary	Clear failure and no ticket issuance.	Fixed-clock boundary tests passed.
Receptionist views and calls	Queue displays ticket; valid current version changes WAITING to CALLED.	Staff queue/call integration test passed.
Patient attempts staff operation	Request is rejected with 403.	Role authorization tests passed.

8.1 Error-message and accessibility review

The page has labelled form controls, keyboard-operable native buttons, and live status regions for dynamic messages. Failure messages are not silent; HTTP error detail is surfaced as text. The retry flow preserves the idempotency key while the client is unsure whether the request completed. This reduces accidental duplicate actions without making a claim that a patient will understand the messages without user testing.

Usability evidence limitation

No actual representative user was recruited, observed or interviewed. No screen-reader or full browser automation was run in this execution. The scripted persona checks and HTML/API smoke checks are preliminary evidence only; moderated task testing remains necessary.

9. Review record and limitations

9.1 Design review checklist

Review question	Finding
Requirements trace to A1/A2?	Yes; see Section 3 and docs/a4/task_contract.md.
Authorization enforced server-side?	Yes; session dependency, role gate and owner check are applied in protected endpoints.
Can duplicate check-in issue another ticket?	No in tested cases; idempotency replay and unique appointment_id constraint prevent it.
Can stale call overwrite current ticket state?	No in tested cases; If-Match plus conditional version update rejects stale requests.
Are writes atomic?	Appointment update, ticket insert and idempotency response are committed together.
Independent human peer review?	Not recorded. An AI-assisted design review and automated checks were performed; neither is a substitute for a human peer.
Real representative-user testing?	Not recorded. Scripted persona walkthrough only; no participant testing claimed.

9.2 Known limits and risks

- Demo credentials are seeded for local coursework use; production must disable demo seeding and integrate an approved identity provider.
- No TLS, rate limiting, multi-factor authentication, audit retention policy, secret rotation or security monitoring is configured in this local prototype.
- SQLite is appropriate for a small demonstrator, not a high-concurrency multi-clinic deployment without further architecture work.
- Queue numbering is based on clinic and date; service-specific queues, doctor assignment, emergency priority and operational exceptions are not modelled.
- Late-arrival handling deliberately directs the patient to reception; the prototype does not automatically mark a no-show or reassign a slot.
- No real clinical, patient or employee data has been used. No clinical workflow has been validated by a clinic.

10. Rollback and recovery guide

The A4 code change is isolated from the A2 domain reference model. The implementation commit is `302a09783bb8810e39c2f694c8927a09330ccde2`, tagged `ise-a4-slice-v1.0`. The evidence snapshot is tagged `ise-a4-v1.0`. The rollback process preserves history and avoids destructive reset.

1. Stop the local Uvicorn process with Ctrl+C.
2. Copy the SQLite database file to a protected backup location before changing code. The coursework database contains synthetic records only.
3. If rolling back the implementation in a shared branch, run `git revert 302a09783bb8810e39c2f694c8927a09330ccde2` and retain the new revert commit.
4. Alternatively, for a separate local rollback test, create a branch from the earlier A2 tag: `git switch -c rollback-a2 ise-a2-v1.0`.

5. Rerun the A2 domain tests and confirm that the prior reference model remains usable.
6. Keep the A4 test logs and database backup for diagnosis. Do not delete persisted data until the owner confirms it is no longer required.

Rollback triggers include unauthorized access, duplicate ticket creation, a stale update overwriting a newer state, state/data inconsistency, or failure to recover clearly after a service interruption.

11. AI-use disclosure

Item	Disclosure
AI tool	ChatGPT (OpenAI).
Tasks	A1/A2 requirement tracing; bounded slice design; code and test drafting; architecture diagrams; report organization.
Advice accepted	Server-side authorization; ownership checks; transaction; idempotency; If-Match concurrency; explicit time-window boundaries and error feedback.
Advice rejected / deferred	Automatic no-show, biometric identification, clinical triage, medical-record integration and automated queue prioritization were excluded because they require broader policy, privacy or clinical governance.
Independent verification	The combined unittest suite was run (60 passed), Python compilation was run, and a local Uvicorn smoke test returned HTTP 200 for the health and UI routes.
AI and review limitation	The code was AI-assisted. No independent human peer review or real participant usability study is claimed.

12. Platform result summary (150–300 words)

This A4 work implements a bounded end-to-end slice of the clinic appointment project: a patient authenticates, creates an appointment with a flexible arrival window, checks in at the clinic, receives one queue ticket, and an authenticated receptionist views and calls that ticket. The browser interface is served by FastAPI; SQLite stores users, sessions, appointments, tickets and idempotency responses. Passwords are stored as PBKDF2 hashes, bearer sessions are stored as token hashes, patient ownership and receptionist roles are checked on the server, and ticket calls use an If-Match version to reject stale updates. Check-in is transactional and uses an idempotency key so an uncertain retry returns the original ticket rather than creating a duplicate. The time interval is half-open: the start is accepted and the end is rejected.

The combined A2 and A4 automated run completed with 60 tests passing: 36 domain-model tests and 24 API integration/flow tests. Coverage includes authorization, ownership, invalid time windows, early and end-boundary check-in, duplicate requests, persistence, missing version headers and stale ticket versions. Synthetic data and a fixed clock make these tests deterministic. The interface route was smoke-tested, and scripted patient/receptionist task walkthroughs were documented. These walkthroughs are not a substitute for moderated testing with real representative users, and no independent human peer review or clinic deployment validation is claimed. The slice excludes clinical decisions, no-show automation, external messaging and medical-record integration.

Word count

229 words.

Appendix A. Changed files and version identity

Artifact	Purpose
app/main.py	FastAPI application, authentication, authorization, SQLite persistence, appointment creation, check-in, queue view and ticket call.
static/index.html	Patient/reception browser interface with status feedback and retry handling.
tests/test_vertical_slice.py	24 A4 HTTP/API integration and end-to-end API-flow checks.
requirements-a4.lock	Pinned runtime/test dependencies for this slice.
models/a4_architecture.*	Architecture diagram source and rendered PNG.
models/a4_call_path.*	End-to-end call-path diagram source and rendered PNG.
docs/a4/	Task contract, usability walkthrough, review record, AI-use log, rollback guide and reproduction notes.
results/a4/	Raw combined test output, environment, smoke test, test summary and development-failure note.

Implementation source commit: 302a09783bb8810e39c2f694c8927a09330ccde2 (tag: ise-a4-slice-v1.0). Implementation change summary: 20 files changed, 831 insertions and 2 deletions. Evidence snapshot tag: ise-a4-v1.0.

Appendix B. Reproduction commands

From the project repository root, use a clean virtual environment and the pinned A4 lock file:

```
python -m venv .venv
# Linux/macOS
. .venv/bin/activate
# Windows PowerShell
# .venv\Scripts\Activate.ps1
python -m pip install -r requirements-a4.lock
python -m unittest discover -s tests -v
python -m uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000. The local demo accounts are `patient / PatientDemo!2026` and `reception / ReceptionDemo!2026`. These are synthetic training credentials and must not be reused or exposed in any deployed service. Do not enter real patient information.

The A2 tests use Python standard-library unittest. A4 uses FastAPI TestClient, HTTPX and temporary SQLite files; all test identities and appointment records are synthetic. The environment recorded for the verified run is in `results/a4/environment.txt`.